HDOC Day-9 Activity

Question 1:

Write a menu-driven C program to input two numbers and swap them using a third variable and without using a third variable. Also display the values before and after swapping.

Step 1: Understanding the Problem

- **Input:** Two integers a and b, and menu choice.
- **Output:** Values before and after swapping.
- **Logic:** Choice 1 uses temp; Choice 2 uses addition and subtraction without a third variable.

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	a=10, b=20, choice=1	After: a=20, b=10	a=20, b=10	Pass
2	a=5, b=9, choice=2	After: a=9, b=5	a=9, b=5	Pass
3	a=-4, b=7, choice=1	After: a=7, b=-4	a=7, b=-4	Pass

Step 3: Pseudocode

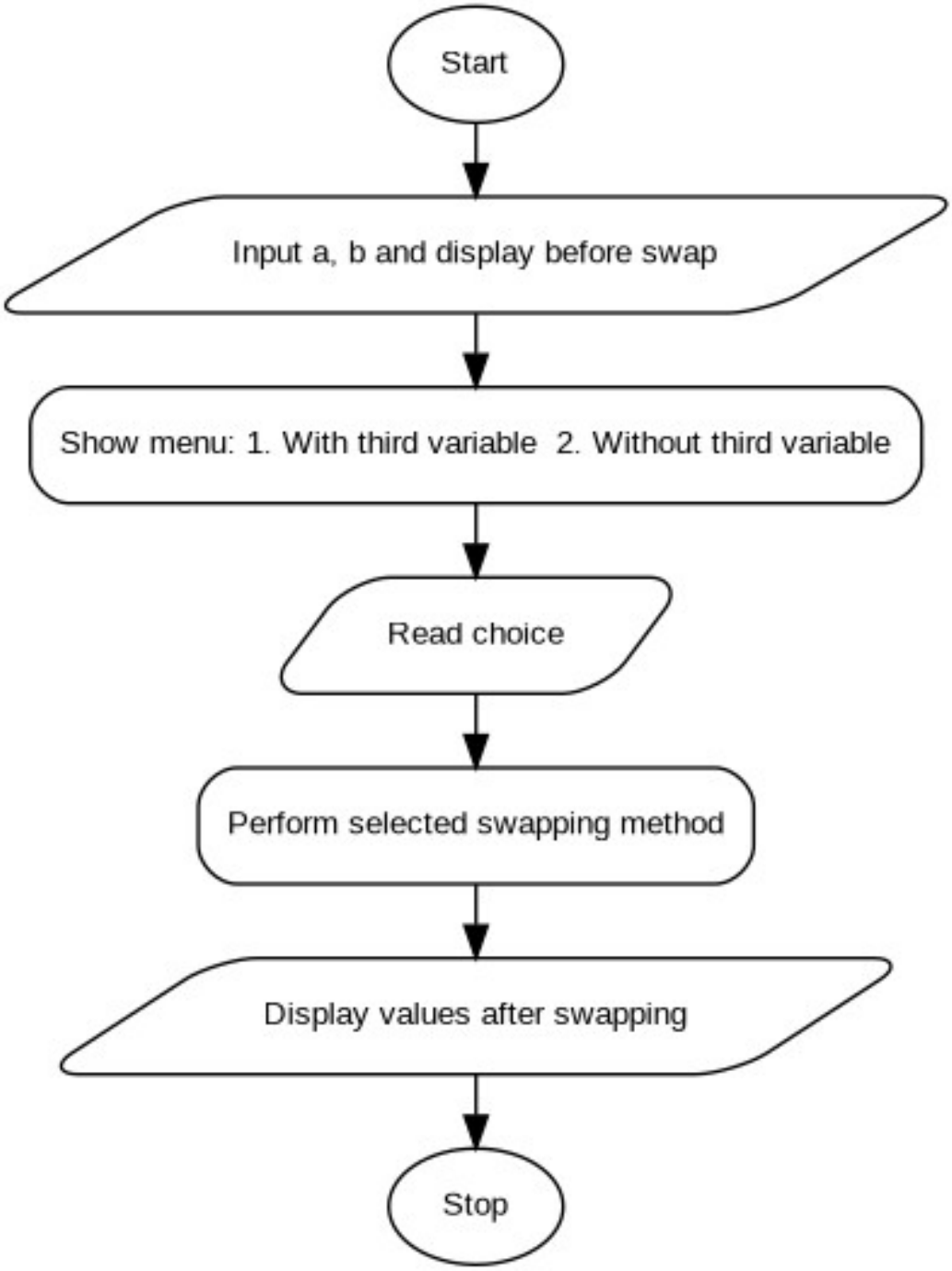
```
START
INPUT a, b
DISPLAY a, b before swapping
DISPLAY menu
INPUT choice
IF choice = 1 THEN
   temp = a; a = b; b = temp
ELSE IF choice = 2 THEN
   a = a + b; b = a - b; a = a - b
ELSE
   DISPLAY "Invalid choice"
   STOP
END IF
DISPLAY a, b after swapping
STOP
```

Step 4: Algorithm and Evaluation

1. Start.
2. Read a and b and display their original values.
3. Display the two swapping choices and read choice.
4. If choice is 1, swap using temp. If choice is 2, swap using arithmetic operations.
5. Display swapped values.
6. Stop.

Evaluation: The algorithm follows the selected menu option and produces the expected result for the prepared test cases.

Step 5: Flowchart and Evaluation



Evaluation: The flowchart represents input, menu selection, calculation, output, and termination in the correct sequence.

Step 6: C Programming

```
#include <stdio.h>
int main()
{
   int a, b, temp, choice;
   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);
   printf("Before swapping: a = %d, b = %d\n", a, b);
   printf("1. Swap using third variable\n");
   printf("2. Swap without third variable\n");
```

```
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1: temp = a; a = b; b = temp; break;
        case 2: a = a + b; b = a - b; a = a - b; break;
        default: printf("Invalid choice\n"); return 0;
    }
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}
```

Step 7: Kali Linux Compilation and Execution

```
(kali@kali)-[~]
$ nano swap.c
(kali@kali)-[~]
$ gcc swap.c -o swap
(kali@kali)-[~]
$ ./swap
Enter two numbers: 10 20
Before swapping: a = 10, b = 20
1. Swap using third variable
2. Swap without third variable
Enter choice: 1
After swapping: a = 20, b = 10
```

Question 2:

Write a menu-driven C program to find the sum of the first n natural numbers and the sum of squares of the first n natural numbers.

Step 1: Understanding the Problem

- **Input:** Positive integer n and menu choice.
- **Output:** Sum of first n natural numbers or sum of their squares.
- **Formula:** Sum = n(n+1)/2; Sum of squares = n(n+1)(2n+1)/6.

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	n=5, choice=1	Sum = 15	Sum = 15	Pass
2	n=5, choice=2	Sum of squares = 55	Sum of squares = 55	Pass
3	n=10, choice=1	Sum = 55	Sum = 55	Pass

Step 3: Pseudocode

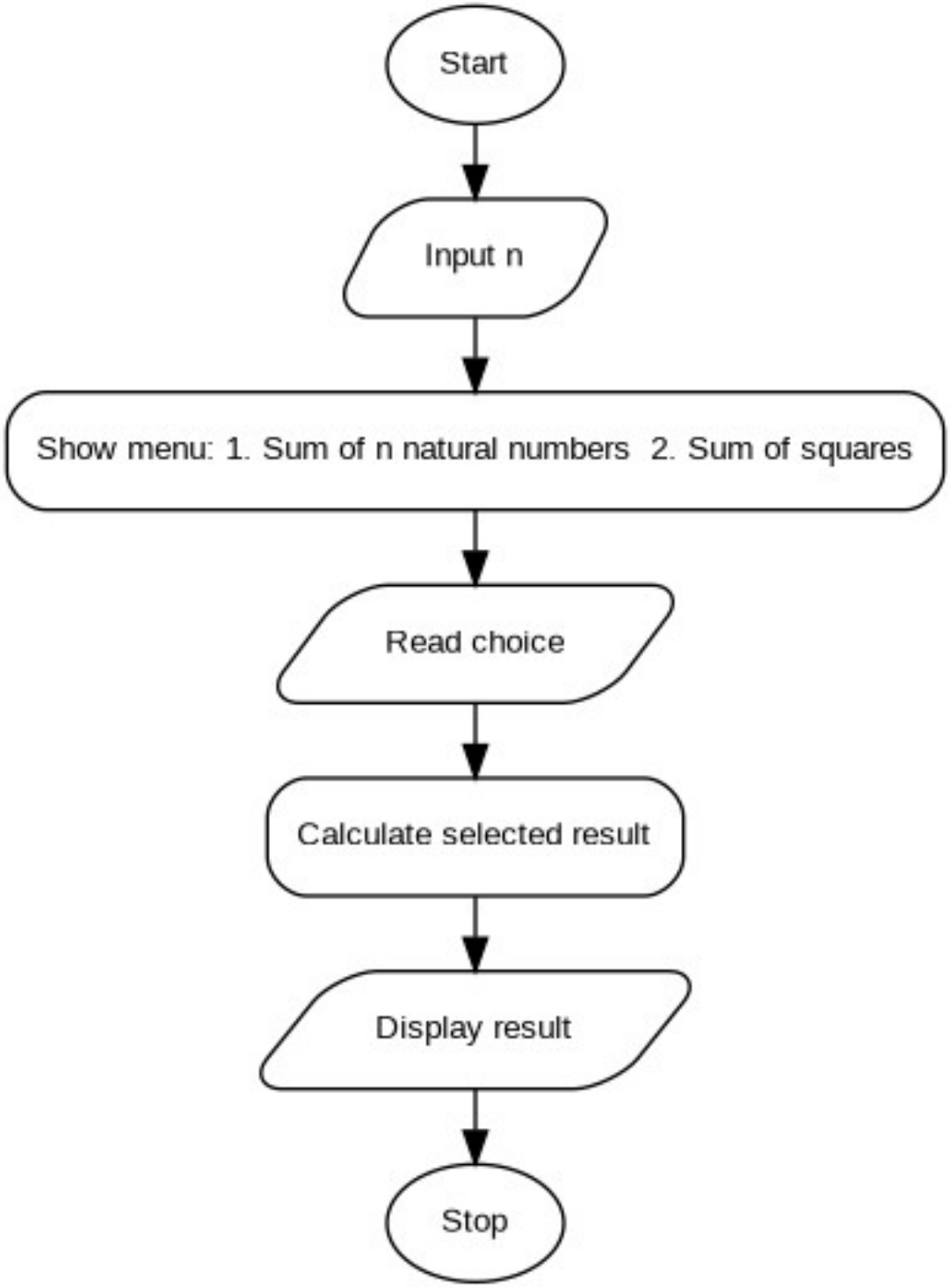
```
START
INPUT n
DISPLAY menu
INPUT choice
IF choice = 1 THEN
   sum = n(n+1)/2
   DISPLAY sum
ELSE IF choice = 2 THEN
   sumsq = n(n+1)(2n+1)/6
   DISPLAY sumsq
ELSE
   DISPLAY "Invalid choice"
END IF
STOP
```

Step 4: Algorithm and Evaluation

7. Start.
8. Read n.
9. Display menu and read choice.
10. For choice 1, calculate n(n+1)/2.
11. For choice 2, calculate n(n+1)(2n+1)/6.
12. Display the selected result.
13. Stop.

Evaluation: The algorithm follows the selected menu option and produces the expected result for the prepared test cases.

Step 5: Flowchart and Evaluation



Evaluation: The flowchart represents input, menu selection, calculation, output, and termination in the correct sequence.

Step 6: C Programming

```
#include <stdio.h>
int main()
{
   int n, choice;
   long long sum, sumsq;
   printf("Enter n: ");
```

```
    scanf("%d", &n);
    printf("1. Sum of first n natural numbers\n");
    printf("2. Sum of squares of first n natural numbers\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    switch(choice)
    {
       case 1:
          sum = (long long)n * (n + 1) / 2;
          printf("Sum = %lld\n", sum);
          break;
       case 2:
          sumsq = (long long)n * (n + 1) * (2 * n + 1) / 6;
          printf("Sum of squares = %lld\n", sumsq);
          break;
       default: printf("Invalid choice\n");
    }
    return 0;
}
```

Step 7: Kali Linux Compilation and Execution

```
(kali@kali)-[~]
$ nano sums.c
(kali@kali)-[~]
$ gcc sums.c -o sums
(kali@kali)-[~]
$ ./sums
Enter n: 5
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter choice: 2
Sum of squares = 55
```

Question 3:

Write a menu-driven C program to calculate simple interest, compound amount, compound interest, and total payable amount for given principal, rate, and time.

Step 1: Understanding the Problem

- **Input:** Principal P, annual rate R, time T, and menu choice.
- **Output:** Selected interest/amount value.
- **Formula:** SI = PRT/100; Compound Amount = P(1+R/100)^T; CI = Amount-P; Total payable (simple-interest option) = P+SI.

Step 2: Standard Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	P=1000,R=10,T=2,ch=1	SI = 200.00	SI = 200.00	Pass
2	P=1000,R=10,T=2,ch=2	Amount = 1210.00	Amount = 1210.00	Pass
3	P=1000,R=10,T=2,ch=3	CI = 210.00	CI = 210.00	Pass

Step 3: Pseudocode

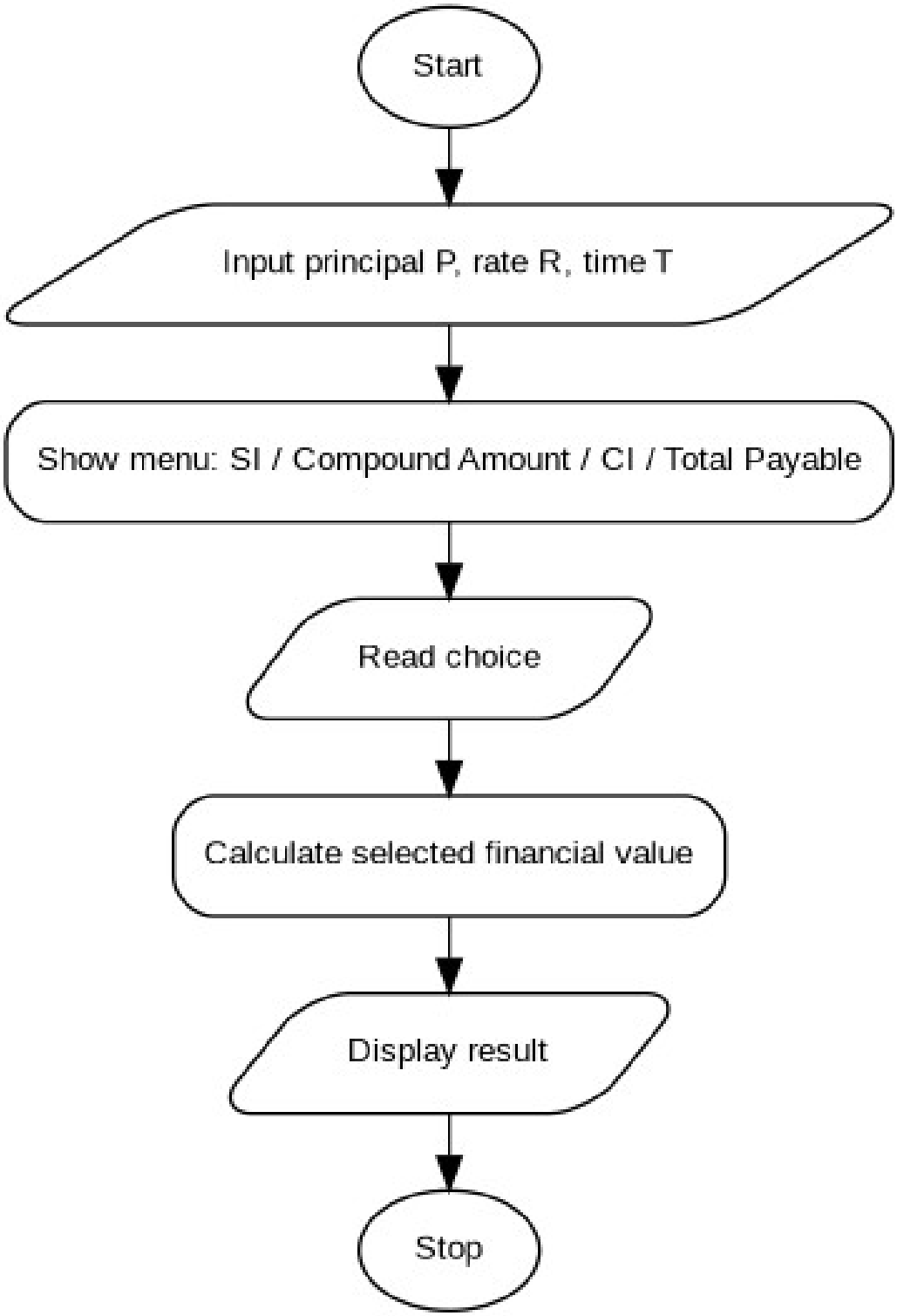
```
START
INPUT P, R, T
DISPLAY menu
INPUT choice
CALCULATE SI, compound amount, CI and total payable
DISPLAY the value selected by choice
IF choice is invalid, DISPLAY error
STOP
```

Step 4: Algorithm and Evaluation

14. Start.
15. Read principal, rate and time.
16. Display menu and read choice.
17. Calculate SI = PRT/100.
18. Calculate compound amount = P(1+R/100)^T and CI = amount - P.
19. Calculate total payable for simple interest as P + SI.
20. Display the selected value.
21. Stop.

Evaluation: The algorithm follows the selected menu option and produces the expected result for the prepared test cases.

Step 5: Flowchart and Evaluation



Evaluation: The flowchart represents input, menu selection, calculation, output, and termination in the correct sequence.

Step 6: C Programming

```
#include <stdio.h>
#include <math.h>
int main()
{
   double p, r, t, si, amount, ci, total;
```

```
    int choice;
    printf("Enter principal, rate and time: ");
    scanf("%lf %lf %lf", &p, &r, &t);
    printf("1. Simple Interest\n2. Compound Amount\n");
    printf("3. Compound Interest\n4. Total Payable Amount\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    si = p * r * t / 100.0;
    amount = p * pow(1.0 + r / 100.0, t);
    ci = amount - p;
    total = p + si;
    switch(choice)
    {
       case 1: printf("Simple Interest = %.2f\n", si); break;
       case 2: printf("Compound Amount = %.2f\n", amount); break;
       case 3: printf("Compound Interest = %.2f\n", ci); break;
       case 4: printf("Total Payable Amount = %.2f\n", total); break;
       default: printf("Invalid choice\n");
    }
    return 0;
}
```

Step 7: Kali Linux Compilation and Execution

```
(kali@kali)-[~]
$ nano interest.c
(kali@kali)-[~]
$ gcc interest.c -o interest -lm
(kali@kali)-[~]
$ ./interest
Enter principal, rate and time: 1000 10 2
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter choice: 3
Compound Interest = 210.00
```